

Block-Diffusion OD Planning — v4 결과 (EOS 수정 백본)

2026-07-27 · 본 보고서는 전적으로 v4 라운드에 기반한다: EOS 경계 수정 후 mask 백본 7개 전체를 재학습하고, 그 위에서 모든 판별자와 평가를 다시 수행했다. 기존 통합 보고서 (RESULTS_BD.pdf)는 수정 전(v2) 백본의 기록으로 보존되며, 여기서는 수정 전/후 비교의 "이전" 측으로만 인용한다. 네이밍: v4 산출물은 전부 _v4 태그를 가진다 — 체크포인트 BD_porto_v3_normal_mask_blk{B}_v4_bd.pth (v3 는 데이터셋 인덱스 porto v3-0.05이며 모델 라운드가 아님), disc BDdisc_*_v4.pth , pool *_pool_v4_* , 평가 태그 v4m* / v4f01m* .

1. EOS 경계 수정

학습 캔버스는 [dst, v0, ..., v_{l-1}=dst, END...END | PAD...] 이고, END는 dst가 속한 블록의 남은 자리에만 채워지며 이후 블록은 PAD(loss 제외)다. 이 스펙에서 dst가 블록 경계에 정확히 떨어지는 샘플은 END 감도가 0이 된다 — 전체 데이터의 1/B 비율이고, B=1이면 모든 샘플이다. 그래서 v2 blk1 백본은 종료를 전혀 학습하지 못했다: unconditional 생성의 100%가 캔버스 캡(길이 101)까지 발산했고 edge의 50.5%가 invalid, valid rate 0.000. B≥2에서는 같은 결함이 경계 종료 길이의 체계적 결핍으로 나타났다: (L+1) mod B = 0인 생성 길이 비율이 실제의 절반 수준(blk2 0.234 vs 0.500; blk32 0.011 vs 0.027).

수정 (bd_models.py build_canvas()): END tail이 비게 되는 경우 캔버스를 한 블록 연장하고 그 블록 전체를 END로 채워 loss에 포함한다. 이제 모든 샘플이 END target을 ≥1개 학습하며, B=1은 샘플마다 정확히 1개다. smoke_test_bd.py 로 검증(캔버스 스펙, 두 커널의 forward/backward-플래닝).

blk64는 구조적으로 불변: 이 데이터셋에 L=63/127인 경로가 사실상 없어(경계 비율 ≈ 0.000) 수정이 blk64 학습 샘플을 하나도 건드리지 않고, 고정 시드 학습이라 v4 blk64는 v2와 동일한 수치를 재현한다. 완전성을 위해 유지한다.

2. Unconditional 생성 — 수정 전/후

백본당 개방 생성 20,000개(plan(None, None) ; ori+dst 동시 dropout 10%로 유효) — 7블록 합계 140,000경로 — 를 NORMAL 세계 기준으로 채점. 실제 참조: 길이 23.2±9.8. valid=모든 edge 유효; invE=invalid edge 비율; KS=길이분포 KS 거리; cap%=캔버스 캡 도달(종료 실패); novel=실제 집합에 그대로 존재하지 않는 비율; bres=경계 잔여류 비율 생성/실제 — 수정 전 END 감도가 결핍됐던 길이 부류. 회색은 v2.

blk	valid v4 (v2)	invE v4 (v2)	len v4 (v2)	KS v4 (v2)	cap% v4 (v2)	novel v4 (v2)	bres 생성/실제 v4 (v2 생성)
1	0.870 (0.000)	2.29% (50.47%)	25.2 (101.0)	0.045 (1.000)	0.9 (100.0)	0.396 (1.000)	—
2	0.737 (0.665)	3.30% (4.41%)	23.3 (24.1)	0.010 (0.042)	0.0 (0.1)	0.427 (0.519)	0.486/0.500 (0.234)
4	0.617 (0.517)	2.59% (3.78%)	23.8 (24.1)	0.026 (0.039)	0.0 (0.1)	0.662 (0.747)	0.235/0.250 (0.173)
8	0.325 (0.382)	5.88% (4.86%)	23.8 (24.5)	0.023 (0.055)	0.0 (0.0)	0.855 (0.834)	0.115/0.125 (0.102)
16	0.202 (0.184)	8.30% (9.09%)	24.4 (23.0)	0.056 (0.019)	0.0 (0.0)	0.902 (0.903)	0.054/0.061 (0.033)
32	0.141 (0.232)	9.37% (11.48%)	25.6 (16.7)	0.120 (0.278)	0.0 (0.0)	0.945 (0.935)	0.032/0.027 (0.011)
64	0.104 (0.104)	11.42% (11.42%)	24.1 (24.1)	0.056 (0.056)	0.0 (0.0)	0.971 (0.971)	0.000/0.000 (0.000)

- blk1 붕괴 완전 해소: valid 0.000 → 0.870 (전 블록 중 최고), invalid edge 50.5% → 2.3%, 길이 101(전량 캡) → 25.2 (KS 0.045). AR에게 필요했던 것은 샘플당 END target 1개였다.
- 경계 잔여류 결핍 소멸: 생성 비율이 실제와 일치(blk2 0.486 vs 0.500; 수정 전 0.234).
- Validity-novelty 트레이드오프는 유지: 작은 블록일수록 valid하지만 학습 경로 복사가 많다(blk1 novel 0.396); blk4-8이 validity 0.33-0.62와 novelty 0.66-0.86의 균형점.
- 유의점: blk8(0.382→0.325), blk32(0.232→0.141)의 valid 하락은 단일 시드 학습 분산과 분리 불가; 다만 blk32의 길이 보정은 크게 개선(KS 0.278→0.120, 평균 16.7→25.6).

2a. 길이 분포와 edge 분포 일치도 (JSEV)

블록당 동일한 생성 20,000개를 전체 실제 NORMAL 집합(최단경로 1,930,710개 — 참조 행)과 비교. KS는 생성/실제 길이 분포 간 Kolmogorov-Smirnov 거리. JSEV는 프로젝트 표준 edge 분포 지표(utils/evaluate_new.py::Evaluator.calculate_divergences)로, 전체 V × V 격자에서 실제/생성 방향 edge 빈도 분포 간 Jensen-Shannon divergence이며 단위는 nats다(scipy rel_entr ; ln 2로 나누면 bits). R²는 동일한 평탄화 edge 분포에서 계산하고, Spearman ρ는 edge 빈도의 순위 상관으로 masked(실제 빈도가 0이 아닌 칸) / 전체 칸 순으로 보고한다. JSEV는 낮을수록, R²·ρ는 높을수록 좋다.

blk	길이 평균 ± 표준편차	중앙값	p5 - p95	KS	JSEV (nats)	R²	Spearman ρ (masked / 전체)
real (1,930,710)	23.21 ± 9.80	23	8 - 40	—	—	—	—
1	25.18 ± 12.51	24	9 - 43	0.045	0.0147	0.985	0.958 / 0.624
2	23.30 ± 9.66	23	8 - 40	0.010	0.0169	0.984	0.972 / 0.579
4	23.76 ± 9.77	23	9 - 40	0.026	0.0147	0.984	0.965 / 0.568
8	23.83 ± 9.85	23	9 - 41	0.023	0.0283	0.973	0.959 / 0.457
16	24.40 ± 9.78	24	9 - 41	0.056	0.0482	0.895	0.931 / 0.420
32	25.61 ± 10.15	26	8 - 42	0.120	0.0576	0.854	0.922 / 0.413
64	24.07 ± 8.94	24	10 - 39	0.056	0.0654	0.875	0.910 / 0.389

- 길이는 전 블록에서 잘 맞는다: 평균이 실제 23.21과 2.4 이내, 중앙값은 3 이내이며 5-95% 폭도 실제 8-40을 근접 추종한다. blk2가 거의 정확하고(23.30 ± 9.66, KS 0.010), blk1은 꼬리가 가장 두껍다(std 12.51), blk32는 길이 보정이 가장 나쁘다(KS 0.120). 참고로 수정 전 blk1 백분은 101 ± 0.00(전량 캡)이었다 — §2 참조.
- edge 분포 충실도는 블록 크기에 따라 단조 악화: JSEV 0.0147(blk1, blk4) → 0.0654(blk64), R² 0.985 → 0.875. 작은 블록이 어떤 도로가 얼마나 자주 쓰이는지를 훨씬 충실히 재현한다.
- 두 Spearman 컬럼이 두 가지 실패 유형을 분리한다: 실제로 등장하는 edge에 한정하면 사용 빈도 순위를 전 블록에서 매우 잘 재현하지만(ρ 0.91-0.97), V² 전체 칸에서는 상관이 훨씬 낮고(0.39-0.62) 블록이 커질수록 떨어진다. 큰 블록이 실제 데이터가 쓰지 않는 edge에 확률 질량을 쓰기 때문이며, §2의 불법 edge 비율(2.3% → 11.4%)과 같은 현상이다.
- edge 분포의 RMSE / MAE도 같은 순서를 따르며(blk1 2.83e-06 / 8.48e-08 → blk64 8.23e-06 / 2.38e-07) `sets_res/uncond_jsev_v4.json`에 기록되어 있다.

2b. Unconditional EM/PC — 각 샘플의 양 끝점을 자신의 OD로

EM/PC는 주어진 OD가 필요 없다: `evaluate_em_pc`는 생성 경로 자신의 (start, end)를 OD로 삼아 참조 최단경로 데이터에서 조회한다. 학습 데이터가 최단경로이므로 **uncond EM** = “샘플이 자기 양 끝점 기준 최단경로인가” — 분포 충실도 지표다. OD 커버리지(생성된 OD가 참조에 존재하는 비율)는 전 블록 ≈1.0이라 측정이 깨끗하다. 블록당 3,000개, seed 7. **novel**∧**EM**이 합성 데이터의 화폐: 알려진 OD의 새로운 최단경로, 즉 데이터셋에 없는 등장(等長) 대안 경로.

blk	valid	EM	PC	novel	valid∧novel	novel∧EM	OD cov
1	0.864	0.640	0.716	0.406	0.271	0.046	1.000
2	0.745	0.629	0.675	0.425	0.170	0.054	1.000
4	0.633	0.401	0.481	0.652	0.285	0.053	0.999
8	0.346	0.178	0.230	0.851	0.197	0.029	0.998
16	0.197	0.113	0.141	0.907	0.104	0.019	0.997
32	0.130	0.058	0.080	0.951	0.081	0.009	0.998
64	0.105	0.035	0.056	0.973	0.078	0.008	0.995

- 최단경로 충실도는 blk1/2가 압도 (EM 0.63-0.64 vs blk4 0.40); novelty-품질 경계에서는 blk2/4가 근소 우위(novel∧EM 0.053-0.054 vs blk1 0.046; valid∧novel blk4 0.285 vs blk1 0.271).
- Novelty 해석 주의: 이 데이터는 최단경로라 OD당 정답이 거의 유일 — 완벽한 생성 모델일수록 novelty가 낮게 나온다. blk1의 복사는 부분적으로 분포를 잘 배운 결과이며, novelty 측 자체가 큰 블록에 유리하게 기울어 있음에도 그 우위는 절대값 ≤1%p다.

2c. Valid 생성물의 다양성 — 직관 검증

원시 novelty는 블록과 함께 상승한다(0.406 → 0.973; 블록 내 mean-field 샘플링의 조합 다양성) — 그러나 invalid 경로의 다양성은 노이즈다. valid 부분집합으로 제한하고 동일 크기 실제 표본과 비교(지표: distinct-OD 비율, 사용된 고유 edge 수, edge 사용 Shannon 엔트로피, edge 집합의 평균 pairwise Jaccard 유사도 — 높을수록 서로 비슷). **n valid**는 그 부분집합의 크기다 — 생성 20,000개 중 모든 edge가 합법인 개수이자 각 행이 계산된 표본 수.

	n valid	distinct OD	edge 지지집합	edge 엔트로피	pairwise Jaccard
real (20k)	20,000	1.000	3,856	10.62 b	0.0124
blk1	17,395	0.995	3,793	10.54 b	0.0132
blk2	14,744	0.996	3,728	10.47 b	0.0151
blk4	12,338	0.995	3,753	10.49 b	0.0131
blk8	6,501	0.996	3,541	10.30 b	0.0149
blk16	4,046	0.995	3,273	10.09 b	0.0189
blk32	2,829	0.995	3,124	9.99 b	0.0186
blk64	2,080	0.998	2,889	10.03 b	0.0186

- Validity로 조건화하면 다양성은 블록 크기와 함께 감소 — 원시 novelty와 정반대: edge 지지집합 3,793 → 2,889, 엔트로피 10.54 → 10.03 b, 샘플 간 유사도 0.0132 → 0.0186. 선택 효과: 큰 B에서는 10-14%만 valid하고, 생존자들은 짧고 흔한 회랑에 응집한다.
- blk1의 valid 집합은 실제 분포의 다양성과 거의 일치 (실제 edge 지지집합의 98%, 엔트로피 -0.08 b, pairwise 유사도 거의 동일). 큰 블록의 겉보기 다양성은 대부분 invalid 노이즈의 다양성이다.
- 블록 크기 선택에 대한 종합: 품질·충실도·유호다양성 모두 소블록 우위; 개방 생성에서 diffusion의 정량적 우위로 남은 것은 blk2-4의 근소한 novel∧EM 마진뿐. (앞서 측정된 샘플링 속도 차이는 알고리즘이 아니라 구현 오버헤드 — first-hitting NFE는 블록 크기와 무관; §10 참조.)

3. Conditional 계획 — held-out 실제 OD 쌍 (normal 세계)

held-out 실제 최단경로 1,000쌍, 생성 시드 3개, mean±std (`eval_bd_sweep_controlled -suffix v4`). arr=목적지 도달; v∧h=valid∧hit. 모두 RAW.

blk	arrival	valid	valid∧hit	EM	PC	s/path
1	0.994±0.002	0.952±0.005	0.948±0.005	0.843±0.007	0.887±0.006	0.082
2	0.948±0.005	0.818±0.006	0.789±0.008	0.756±0.005	0.782±0.006	0.060
4	0.937±0.004	0.878±0.008	0.826±0.005	0.786±0.010	0.823±0.010	0.035

8	0.932±0.012	0.728±0.003	0.687±0.010	0.671±0.004	0.695±0.003	0.030
16	0.926±0.000	0.676±0.013	0.641±0.008	0.600±0.010	0.631±0.011	0.031
32	0.879±0.004	0.585±0.010	0.529±0.011	0.499±0.006	0.533±0.008	0.032
64	0.839±0.005	0.430±0.014	0.373±0.010	0.366±0.012	0.391±0.012	0.033

Conditional 품질은 여전히 소블록 우위(EOS 수정은 dst first-hit 중요를 건드리지 않음). §2와 합치면 역할 구분이 명확하다: conditional 계획 → 소블록: unconditional/개방 합성 → diffusion 블록 (blk1은 수정 전에는 그 용도로 불가능했고 수정 후 경쟁력이 생겼지만, novelty와 guidance 여지는 큰 블록이 우세).

4. 예외 시나리오 guidance — fam 0.05 (except_0, RAW)

except_0 예약 1,000쌍(어떤 disc도 학습에 사용 안 함), seed 7, first-hitting, 블록별 model-negative disc(_v4), n_is=100. 방법별 컬럼: valid / arrival / valid∧arrival / EM / PC, 모두 RAW. 기본 IW는 표에서 생략(모든 블록에서 base < IW < adj+IW 순서 유지; 예: blk4 valid 0.279/0.332/0.389). Seen=except_0(1%, 예약 행 제외)으로 학습; Unseen=except 1-99로 학습 후 zero-shot 적용.

4.1 Seen

blk	base valid	base arr	base v∧a	base EM	base PC	adj+IW valid	adj+IW arr	adj+IW v∧a	adj+IW EM	adj+IW PC
1	0.302	0.994	0.300	0.271	0.283	0.928	0.567	0.540	0.374	0.425
2	0.251	0.949	0.238	0.233	0.240	0.492	0.846	0.401	0.388	0.423
4	0.279	0.932	0.262	0.243	0.258	0.389	0.873	0.328	0.334	0.357
8	0.229	0.938	0.214	0.202	0.214	0.331	0.862	0.273	0.295	0.309
16	0.206	0.923	0.194	0.180	0.190	0.298	0.865	0.250	0.230	0.255
32	0.185	0.870	0.160	0.157	0.168	0.265	0.825	0.215	0.219	0.237
64	0.141	0.827	0.119	0.129	0.134	0.229	0.795	0.162	0.176	0.198

4.1b IW 단독 — 인접성 마스킹 없음, seen

§4.1과 동일 판별자에 Lemma-3 제안 마스킹만 끈 설정으로, 판별자 재가중 자체의 기여를 분리한다. 컬럼 동일, 모두 RAW.

blk	base valid	base arr	base v∧a	base EM	base PC	IW valid	IW arr	IW v∧a	IW EM	IW PC
1	0.302	0.994	0.300	0.271	0.283	0.324	0.991	0.321	0.283	0.298
2	0.251	0.949	0.238	0.233	0.240	0.328	0.951	0.312	0.306	0.315
4	0.279	0.932	0.262	0.243	0.258	0.332	0.936	0.304	0.295	0.310
8	0.229	0.938	0.214	0.202	0.214	0.294	0.923	0.254	0.270	0.279
16	0.206	0.923	0.194	0.180	0.190	0.260	0.917	0.236	0.211	0.230
32	0.185	0.870	0.160	0.157	0.168	0.244	0.867	0.207	0.204	0.220
64	0.141	0.827	0.119	0.129	0.134	0.201	0.854	0.157	0.160	0.177

4.2 Unseen (except 1-99 → except_0; base는 §4.1과 동일)

blk	base valid	base arr	base v∧a	base EM	base PC	adj+IW valid	adj+IW arr	adj+IW v∧a	adj+IW EM	adj+IW PC
1	0.302	0.994	0.300	0.271	0.283	0.925	0.558	0.530	0.362	0.412
2	0.251	0.949	0.238	0.233	0.240	0.488	0.848	0.415	0.367	0.408
4	0.279	0.932	0.262	0.243	0.258	0.390	0.875	0.331	0.335	0.357
8	0.229	0.938	0.214	0.202	0.214	0.331	0.863	0.273	0.295	0.308
16	0.206	0.923	0.194	0.180	0.190	0.300	0.868	0.254	0.237	0.261
32	0.185	0.870	0.160	0.157	0.168	0.268	0.828	0.219	0.222	0.240
64	0.141	0.827	0.119	0.129	0.134	0.224	0.798	0.162	0.178	0.197

4.2b IW 단독 — 인접성 마스킹 없음, unseen

§4.1b와 동일하되 zero-shot(except 1-99) 판별자를 쓴다.

blk	base valid	base arr	base v∧a	base EM	base PC	IW valid	IW arr	IW v∧a	IW EM	IW PC
-----	------------	----------	----------	---------	---------	----------	--------	--------	-------	-------

1	0.302	0.994	0.300	0.271	0.283	0.321	0.992	0.319	0.281	0.296
2	0.251	0.949	0.238	0.233	0.240	0.330	0.951	0.316	0.305	0.316
4	0.279	0.932	0.262	0.243	0.258	0.331	0.935	0.302	0.295	0.310
8	0.229	0.938	0.214	0.202	0.214	0.298	0.923	0.258	0.275	0.284
16	0.206	0.923	0.194	0.180	0.190	0.264	0.919	0.242	0.215	0.234
32	0.185	0.870	0.160	0.157	0.168	0.246	0.869	0.209	0.208	0.223
64	0.141	0.827	0.119	0.129	0.134	0.199	0.857	0.157	0.161	0.177

- v2 컬론이 수정된 백본에서 재현: adj+IW는 모든 블록에서 엄격한 raw 성공률(valid^arrival)을 개선하며, arrival을 내주고 validity를 크게 얻는다; blk1이 극단(valid 0.302→0.928, arrival 0.994→0.567, v^a 0.300→0.540).
- Seen/unseen 격차 ≤0.015 (v^a blk1 0.540/0.530, blk4 0.328/0.331) — zero-shot 시나리오 전이 재확인; 판별자 수준 분석은 **DISC_VALIDITY_K0.pdf**.
- adj+IW ESS ≈ 95.5(blk64)-99.7(blk1): reveal당 disc 개입은 완만하며, blk1의 상승은 disc가 아니라 Lemma-3 인접성 마스킹이 견인.

5. 공개 베이스라인과의 통제 비교: D-CBG (Schiff et al.)

*Simple Guidance Mechanisms for Discrete Diffusion Models*의 플러그인 이식(`dcbg_plugin.py` , 그들의 `_cbg_denoise` 로직 그대로)을 v4 백본에서 재실행. IW와 다른 점은 방법론 고유의 차이뿐: D-CBG는 노이즈 상태 $p_\phi(y | x_t, t)$ 를 채점해 위치별 categorical을 밀팅하고, IW는 *clean* 부분경로를 채점해 joint 후보를 재가중한다. 나머지는 모두 통제 일치 — 동일한 인접행렬 인자 아키텍처(`-adj 1`), 동일한 1% 데이터 예산·split seed, 동일한 model-negative(각 블록 자신의 v4 unconditional pool을 모델 forward process로 노이즈 처리), 동일한 except_0 예약 1,000쌍, seed 7.

양쪽 모두 $\gamma = 1$. 이전 라운드는 D-CBG만 $\gamma = 4$, IW는 기본값 $\gamma = 10$ 이라 서로 다른 강도에서 비교한 셈이었다. 여기서는 둘 다 tempering 없는 정확한 형태다 — IW의 $w_n = \exp(\gamma \cdot \ell_\phi)$ 는 밀도비 $D/(1-D)$ 그대로가 되고, D-CBG의 $\log p_\theta + \gamma \cdot \log p_\phi$ 는 Bayes 필터 그대로가 된다. (참고로 v2 백본에서의 v1-era γ 스위치: D-CBG blk4 valid $\gamma=1/2/4$ 에서 0.262 / 0.273 / 0.289 — 이 방법은 $\gamma \gg 1$ 이라야 움직인다.)

exact vs first-order는 동일 분류기에 대한 런타임 플러그(`-approx`)이지 별도 모델이 아니다. exact: reveal 위치에서 $V = 1,393$ 개 토큰을 전수 열거 — first-hitting에서는 스텝당 그 한 자리만 소비되므로 근사가 아니라 정확하다. first-order: 그들의 Taylor 변형, reveal당 one-hot 입력에 대한 forward+backward 1회.

5.1 Seen (except_0으로 분류기 학습) — §4.1과 동일 컬럼, 모두 RAW

blk	base valid	base arr	base v^a	base EM	base PC	D-CBG valid	D-CBG arr	D-CBG v^a	D-CBG EM	D-CBG PC
1	0.302	0.994	0.300	0.271	0.283	0.324	0.995	0.321	0.283	0.298
2	0.251	0.949	0.238	0.233	0.240	0.278	0.946	0.264	0.255	0.264
4	0.279	0.932	0.262	0.243	0.258	0.298	0.939	0.281	0.258	0.275
8	0.229	0.938	0.214	0.202	0.214	0.239	0.927	0.219	0.208	0.222
16	0.206	0.923	0.194	0.180	0.190	0.226	0.917	0.207	0.197	0.208
32	0.185	0.870	0.160	0.157	0.168	0.206	0.872	0.174	0.170	0.184
64	0.141	0.827	0.119	0.129	0.134	0.156	0.813	0.126	0.132	0.142

5.2 Unseen (except 1-99 학습 → except_0 zero-shot; base는 §5.1과 동일)

blk	base valid	base arr	base v^a	base EM	base PC	D-CBG valid	D-CBG arr	D-CBG v^a	D-CBG EM	D-CBG PC
1	0.302	0.994	0.300	0.271	0.283	0.322	0.994	0.319	0.279	0.295
2	0.251	0.949	0.238	0.233	0.240	0.280	0.948	0.266	0.255	0.265
4	0.279	0.932	0.262	0.243	0.258	0.299	0.937	0.280	0.256	0.274
8	0.229	0.938	0.214	0.202	0.214	0.243	0.931	0.222	0.211	0.225
16	0.206	0.923	0.194	0.180	0.190	0.225	0.920	0.207	0.194	0.205
32	0.185	0.870	0.160	0.157	0.168	0.196	0.869	0.166	0.162	0.175
64	0.141	0.827	0.119	0.129	0.134	0.151	0.820	0.125	0.130	0.139

5.3 First-order (Taylor) 변형, seen — §5.1과 동일 컬럼

§5.1/§5.2와 동일 분류기에 `-approx 1` : reveal당 $V = 1,393$ 개 토큰 전수 열거 대신 one-hot 입력에 대한 forward+backward 1회. 모두 RAW.

blk	base valid	base arr	base v^a	base EM	base PC	D-CBG (1st-order) valid	D-CBG (1st-order) arr	D-CBG (1st-order) v^a	D-CBG (1st-order) EM	D-CBG (1st-order) PC
1	0.302	0.994	0.300	0.271	0.283	0.301	0.994	0.299	0.271	0.283

2	0.251	0.949	0.238	0.233	0.240	0.251	0.948	0.237	0.235	0.242
4	0.279	0.932	0.262	0.243	0.258	0.281	0.930	0.262	0.246	0.261
8	0.229	0.938	0.214	0.202	0.214	0.228	0.936	0.213	0.202	0.214
16	0.206	0.923	0.194	0.180	0.190	0.206	0.922	0.194	0.181	0.191
32	0.185	0.870	0.160	0.157	0.168	0.184	0.871	0.159	0.155	0.166
64	0.141	0.827	0.119	0.129	0.134	0.142	0.825	0.120	0.130	0.135

5.4 First-order (Taylor) 변형, unseen — §5.2와 동일 컬럼

blk	base valid	base arr	base v^a	base EM	base PC	D-CBG (1st-order) valid	D-CBG (1st-order) arr	D-CBG (1st-order) v^a	D-CBG (1st-order) EM	D-CBG (1st-order) PC
1	0.302	0.994	0.300	0.271	0.283	0.302	0.994	0.300	0.271	0.283
2	0.251	0.949	0.238	0.233	0.240	0.252	0.948	0.239	0.234	0.241
4	0.279	0.932	0.262	0.243	0.258	0.282	0.930	0.265	0.246	0.261
8	0.229	0.938	0.214	0.202	0.214	0.230	0.934	0.214	0.203	0.215
16	0.206	0.923	0.194	0.180	0.190	0.207	0.923	0.195	0.182	0.192
32	0.185	0.870	0.160	0.157	0.168	0.183	0.870	0.158	0.155	0.166
64	0.141	0.827	0.119	0.129	0.134	0.141	0.824	0.119	0.129	0.134

전 블록-양 방식에서 inert: validity가 base와 ±0.003 이내이고(blk1 0.301 vs 0.302; blk4 0.281 vs 0.279; blk64 0.142 vs 0.141), v^a / EM / PC도 마찬가지다. §5.1의 exact 변형(blk4 0.298, blk64 0.156)과 대조된다. v2 백본에서의 v3 보고서 결론이 그대로 재현된 것으로 — one-hot 입력에서의 gradient는 실제 토큰 치환의 효과를 근사하지 못한다. 비용은 ≈0.013-0.019 s/path(§5.6)로 가장 싼 guided 옵션이자 유일하게 얻는 것이 없는 옵션이다.

5.5 다섯 방법 한눈에 비교 — γ = 1, raw validity

§4.1과 §5.1의 수치를 한 표로 모은 것 — base, D-CBG 두 변형, IW 두 변형이 동일 guidance 강도에서 함께 놓이는 유일한 표다.

blk	base	D-CBG first-order	D-CBG exact	IW	adj+IW
1	0.302	0.301	0.324	0.324	0.928
2	0.251	0.251	0.278	0.328	0.492
4	0.279	0.281	0.298	0.332	0.389
8	0.229	0.228	0.239	0.294	0.331
16	0.206	0.206	0.226	0.260	0.298
32	0.185	0.184	0.206	0.244	0.265
64	0.141	0.142	0.156	0.201	0.229

- 동일 강도에서 IW가 blk2 이상 전 구간에서 D-CBG를 앞선다: base 대비 exact D-CBG는 +1.0~2.7%p, IW는 +2.2~7.7%p, adj+IW는 그 이상. 순서 base < D-CBG(fo) ≈ base < D-CBG(exact) < IW < adj+IW가 전 블록에서 유지된다.
- blk1은 **동일(0.324)**이고 그 이유가 측정된다: 생성 경로를 1:1로 비교하면 D-CBG는 base 대비 5%만 바꾸고(950/1000 동일), IW는 18%를 바꾼다(824/1000 동일). B = 1에서는 블록 내 후보 다양성이 없어(ESS ≈ 99.7) 어떤 재가중도 크게 작용할 수 없다 — §4의 blk1 이득은 분류기가 아니라 Lemma-3 인접성 마스킹에서 나온다.
- first-order 변형은 v4에서도 inert(§5.3-§5.4): 전 블록-양 방식에서 base와 ±0.003 이내 — v2 백본에서의 v3 보고서 결론이 그대로 재현된다.
- D-CBG도 zero-shot 전이된다(seen/unseen 격차 전 지표 ≤0.010). 즉 §8의 일반화 결론은 IW 판별자 고유의 성질이 아니라, 시나리오 인접행렬을 입력으로 주는 설계에서 따라온다.
- 제거 edge 사용률은 base 수준에 머문다(~4.9-5.1% vs base ~5%) — 위치별 토크는 시나리오 제약을 거의 인코딩하지 못하며, 그 수치를 ~3%로 낮추는 것은 IW의 Lemma-3 마스킹이다.

5.6 실행 시간과 채점 호출 수 — 동일 조건 측정

타이머가 정확히 재는 것: OD 쌍 100개짜리 배치에 대한 플래너 호출 1회의 wall-clock — (origin, destination) 리스트를 넘긴 시점부터 완성된 정점 경로를 돌려받는 시점까지 — 을 100으로 나눈 값이다. 포함: denoiser forward, guidance 채점(판별자/분류기), 후보 샘플링, reveal별 bookkeeping, 플래너 자체의 CPU sync-정지 스캔, 경로 조립. 제외: 체크포인트/데이터 로딩, P1/P3 후처리, EM/PC/DTW 채점, 그리고 플래너가 반환한 이후의 모든 것. 따라서 이 값은 **100경로를 동시에 계획할 때의 경로당 분할 비용**이며, 경로 하나를 단독으로 계획할 때의 지연이 $O(L/C)$ (그 경우 GPU 활용도가 낮아 경로당 비용은 훨씬 커진다).

공정성 통제(time_guidance.py): 모든 방법에 대해 동일 OD 쌍-동일 배치-동일 시도; 한 번에 한 방법씩 순차 실행, 스크립트가 유효임을 확인한 GPU 하나에서만 — nvidia-smi 로 상주 compute 프로세스를 조회하고 하나라도 있으면 실행을 거부한다; (블록, 방법) 칸마다 warm-up 배치를 1회 실행하고 버려서 CUDA 컨텍스트 생성과 커널 오버헤드가 보고 수치에 절대 섞이지 않게 했다; 모든 측정 구간은 torch.cuda.synchronize() 로 감쌌다; 그리고 칸마다 예약 풀의 서로 다른 슬라이스 3배치(쌍 1-100, 101-200, 201-300)를 측정하며 이는 방법 간 동일하다. 그 3배치의 평균 ± 표준편차로 보고하므로, 산포는 실행 간 지터뿐 아니라 배치 구성 차이도 반영한다. 칸당 300경로, 총 35칸.

비교 불가 대상: (i) 평가 실행이 blk4 D-CBG-exact에 대해 출력한 0.63 s/path — 이는 eval_dcbg.py 의 sec_per_path 로, 타이머가 생성에 더해 P1/P3 스플라이싱과 EM/PC 채점까지 포함하고 4개 프로세스가 CPU를 경합한 값이며, 여기 계획 단독 0.328과 대비된다; (ii) §3의 s/path 컬럼 — eval_bd_sweep_controlled.py 값으로 생성에 더해 1,000경로에 대한 EM/PC/DTW-길이 채점까지 포함한다. 채점 호출 = 경로당 판별자/분류기 시퀀스 평가 횟수로, 구현과 무관한 비용이다(실제 경로에서 공개 토큰 $L \approx 22$): $IW = L \cdot \eta_{ls}$, $D-CBG\ exact = L \cdot V$, first-order = $L \cdot H$ forward+backward.

blk	base	IW	adj+IW	D-CBG first-order	D-CBG exact
1	0.0050 ± 0.0002	0.0264 ± 0.0028	0.0934 ± 0.0000	0.0132 ± 0.0005	0.3126 ± 0.0242
2	0.0044 ± 0.0003	0.0262 ± 0.0026	0.0531 ± 0.0285	0.0127 ± 0.0007	0.3052 ± 0.0323
4	0.0043 ± 0.0002	0.0277 ± 0.0018	0.0278 ± 0.0018	0.0130 ± 0.0006	0.3281 ± 0.0222
8	0.0045 ± 0.0000	0.0315 ± 0.0000	0.0316 ± 0.0000	0.0135 ± 0.0001	0.3768 ± 0.0000
16	0.0056 ± 0.0000	0.0435 ± 0.0000	0.0437 ± 0.0000	0.0156 ± 0.0000	0.5313 ± 0.0001
32	0.0060 ± 0.0000	0.0517 ± 0.0000	0.0520 ± 0.0000	0.0158 ± 0.0000	0.6385 ± 0.0001
64	0.0072 ± 0.0000	0.0669 ± 0.0000	0.0676 ± 0.0000	0.0163 ± 0.0000	0.8360 ± 0.0002
채점 호출 / 경로	0	≈2,200	≈2,200	≈22 (fwd+bwd)	≈30,600

s/path, 서로 다른 100-배치 3개에 대한 평균 ± 표준편차. 배치당 reveal 스텝 수(blk1 / blk4 / blk64): IW 48 / 50 / 62, adj+IW 100 / 50 / 62.

- Exact D-CBG는 IW의 12배 비용인데 이득은 더 작다(blk4: 0.328 vs 0.028 s/path; blk64: 0.836 vs 0.067). 격차는 구조적이다 — reveal당 V = 1,393개 치환을 전부 채점하는 반면 IW는 η_{is} = 100개 표본 후보만 채점한다.
- first-order는 가장 싼 guided 옵션(0.013-0.016 s/path)이지만 얻는 게 없다 — IW보다 싸지만 통계적으로 unguided base와 구분되지 않는다(\$5.3-\$5.4).
- base 계획은 블록 크기에 거의 평평하다(0.0043-0.0072 s/path) — first-hitting은 B와 무관하게 공개 토큰당 forward 1회를 쓰고, 늘어나는 캔버스 길이만 비용을 더한다. 두 guided 계열도 같은 이유로 완만히 증가한다. 예외는 blk1/2의 adj+IW(0.093 / 0.053 s/path): 인접성 마스킹 탓에 목적지를 지나치는 경로가 많아 배치당 reveal이 늘어난다(blk1에서 100 vs 48) — \$4.1에서 arrival 0.567로 나타나는 바로 그 효과다. blk2의 큰 표준편차(±0.0285)도 같은 불안정성으로, 세 배치가 각각 100 / 60 / 54 reveal이었다. 전체 분해는 \$5.6.2.
- blk8 이상에서의 ±0.0000 표준편차는 실제 값이며 전사 오류가 아니다. 배치별 원시 wall-clock을 출력해 확인했다: blk8 IW = 3.134 / 3.138 / 3.138 s, blk16 D-CBG-exact = 53.246 / 53.247 / 53.244 s. 배치 100에서는 어떤 쌍이 들어오든 reveal 스텝 수가 고정된다 — 플레너는 배치 전체가 멈출 때까지 돌고, 100경로면 그중 최장 경로가 항상 풀 최댓값 근처이기 때문이다 — 그래서 wall-clock이 0.1% 미만으로 결정적이다. blk1-4에서는 배치마다 reveal 수가 아직 달라지고, 0이 아닌 표준편차는 거기서 나온다.
- first-order 컬럼에 대한 주의: 프로세스 간에 ±20% 변동한다(blk8: 본 실행 0.0135 vs 재실행 0.0166; blk16: 0.0156 vs 0.0188). 이 칸은 연산이 아니라 커널 실행 오버헤드에 묶여 있어 드라 이버의 다른 작업에 민감하다. 소수 4자리가 아니라 ≈0.013-0.019로 읽어야 한다.
- 주의: wall-clock은 본 구현(KV 캐시·블록 내 병렬 확정 미구현)과 이 GPU에 종속된다. 이식 가능한 비교는 채점 호출 행이다.

5.6.1 배치 = 100이 특정 방법에 불리한가?

IW는 내부적으로 경로당 η_{is} = 100개 후보를 돌리므로, 배치 100이면 reveal당 이미 10^4 개 시퀀스를 채점한다 — 이 천장이 계속 분할이익을 얻는 방법들에 비해 IW를 불리하게 만드는지가 쟁점이다. 동일 프로토콜, blk4, 배치 25 → 400 스위치(여기서는 칸당 측정 배치 1개라 표준편차 없음):

batch	base	IW	adj+IW	D-CBG first-order	D-CBG exact
25	0.0140	0.0333	0.0338	0.0542	0.2653
50	0.0077	0.0284	0.0287	0.0292	0.2889
100	0.0044	0.0277	0.0279	0.0157	0.3285
200	0.0032	0.0263	0.0276	0.0084	0.3274
400	0.0030	0.0304	0.0285	0.0055	0.3599

- IW는 배치 크기에 평평하다(25 → 400에서 0.0333 → 0.0304, 단조 추세 없음) — η_{is} = 100에서 이미 연산 바운드라 배치 100이 손해를 주지 않는다. adj+IW(0.0338 → 0.0285)와 D-CBG exact(0.265 → 0.360, 오히려 큰 배치에서 소폭 약화)도 마찬가지다.
- 배치 = 100이 실제로 불리하게 만드는 것은 D-CBG first-order로, 끝까지 분할이익을 얻는다: bs 25에서 0.0542 → bs 400에서 0.0055, 즉 본 표가 쓴 bs 100 대비 bs 400에서 2.9배 빠르 다. 따라서 본 표는 큰 배치에서의 first-order 속도를 과소평가한다 — 다만 결론은 바뀌지 않는다. first-order의 문제는 느린 것이 아니라 아무것도 바꾸지 못한다는 것이기 때문이다(\$5.3).
- base도 계속 분할이익을 얻는다(0.0140 → 0.0030). bs 400(blk4)에서 OOM이 난 방법은 없었으므로, 배치가 IW의 메모리 사용량 때문에 제한된 것도 아니다.

5.6.2 adj+IW가 블록 크기에 대해 U자형인 이유 (blk4가 최소)

adj+IW는 단조가 아닌 유일한 컬럼이다: 0.093 → 0.053 → **0.028** → 0.032 → 0.044 → 0.052 → 0.068. 이는 block diffusion의 성질이 아니라, 서로 반대 방향으로 움직이는 두 요인의 곱이며 둘 다 샘플러에서 직접 확인된다. 본 구현에는 KV 캐시가 없어서 매 reveal 스텝마다 denoiser가 현재 캔버스 전체를 다시 계산하고, 판별자도 그 길이의 후보 시퀀스 η_{is} 개를 채점한다. 따라서 배치 비용 은 1차 근사로 $\sum_{reveals}$ (그 reveal 시점의 캔버스 길이)이다 — 이를 토큰-forward 수라 하자. reveal 스케줄은 블록 크기와 배치가 멈춘 블록이 정해지면 결정적이므로, 이 값을 정확히 계산해 실측 시 계와 대조할 수 있다:

blk	reveal 스텝		forward당 평균 캔버스		토큰-forward Σs		실측 s/path		토큰-forward당 μs	
	IW	adj+IW	IW	adj+IW	IW	adj+IW	IW	adj+IW	IW	adj+IW
1	49.7	100.0	27.4	52.5	1.36k	5.25k	0.0264	0.0934	19.4	17.8
2	50.0	71.3	28.1	41.6	1.40k	2.97k	0.0262	0.0531	18.6	17.9
4	51.3	51.3	29.7	29.7	1.52k	1.52k	0.0277	0.0278	18.2	18.3
8	54.0	54.0	32.9	32.9	1.78k	1.78k	0.0315	0.0316	17.7	17.8
16	62.0	62.0	40.8	40.8	2.53k	2.53k	0.0435	0.0437	17.2	17.3

32	62.0	62.0	48.5	48.5	3.01k	3.01k	0.0517	0.0520	17.2	17.3
64	62.0	62.0	64.0	64.0	3.97k	3.97k	0.0669	0.0676	16.9	17.0

reveal 수는 동일한 3배치에서 실측한 배치별 값(`dbg_reveals.py`), Σs 는 reveal 스케줄에서 계산, 마지막 쌍은 실측 시간 $\div \Sigma s$.

- 마지막 두 컬럼은 14칸 전체에서 17-19 μs 로 평평하다($\pm 7\%$). 상수 하나 $\times$ 토큰-forward 수가 U자형을 포함한 전체 곡면을 재현한다 — 즉 블록 크기 고유의 설명이 필요한 것이 아니라 아래 두 요인만 있으면 된다.
- 왼쪽 팔(blk1→4): 배치가 멈추지 않는다. 플래너는 모든 행이 목적지에 도달하거나 END를 내놓을 때까지 돌므로, 낙오 행 하나가 100행 전체의 wall-clock을 결정한다. blk1에서 Lemma-3 인접성 마스크를 걸면 100행 중 35-38행이 아예 종료하지 않고 세 배치 모두 캔버스 상한까지 돈다(reveal 100 vs IW 48) — \$4.1의 arrival 0.567을 타이머 쪽에서 본 것이다. blk2에서는 낙오 행이 결국 멈추기는 하지만 매우 늦고 일관되지 않다: 세 배치가 각각 100 / 60 / 54 reveal이었고, 본 표의 ± 0.0285 표준편차는 정확히 여기서 나온다. blk4부터는 reveal 수가 순수 IW와 동일해져 (50 / 50 / 54) adj+IW 비용이 IW와 소수 4자리까지 같아진다.
- 오른쪽 팔(blk4→64): forward마다 캔버스가 더 길다. reveal 수는 51 → 62로 1.2배만 늘지만, forward당 평균 캔버스는 30 → 64로 2.2배 늘어난다. 이유는 둘이다: (i) 캔버스는 한 번에 블록 하나씩 늘어나므로, B = 64면 62번의 reveal이 모두 길이 64에서 돌지만 B = 4면 초반 reveal은 길이 4, 8, 12...에서 돈다; (ii) 정지 판정은 블록 경계에서만 이뤄지므로 큰 블록은 초과 진행한다 — blk4가 52에서 멈추는 데 반해 blk16/32/64는 모두 캔버스 64까지 간다.
- 함의: blk4 최소값은 두 효과가 교차하는 지점이며, 방법론이 아니라 구현의 산물이다. KV 캐시를 쓰면 오른쪽 팔은 거의 평평해지고(reveal당 증분 비용이 캔버스 길이에 따라 커지지 않는다) 왼쪽의 낙오 효과만 남는다. \$5.6의 채점 호출 행은 둘 중 어느 것에도 영향받지 않는다.

6. Guidance — fam 0.1 (except_0, RAW; 동일 프로토콜)

edge 제거 비율 0.1의 동일 파이프라인: 제거 edge가 2배인 훨씬 어려운 세계 — base validity가 0.04-0.12로 하락. fam 0.1 시나리오로 학습한 블록별 `_v4` disc; unseen = except 1-99 → except_0 zero-shot.

6.1 Seen

blk	base valid	base arr	base $v \wedge a$	base EM	base PC	adj+IW valid	adj+IW arr	adj+IW $v \wedge a$	adj+IW EM	adj+IW PC
1	0.115	0.993	0.114	0.108	0.110	0.927	0.250	0.234	0.144	0.177
2	0.096	0.960	0.092	0.089	0.092	0.245	0.717	0.163	0.174	0.200
4	0.089	0.949	0.084	0.075	0.081	0.169	0.841	0.137	0.141	0.152
8	0.077	0.933	0.071	0.073	0.074	0.143	0.804	0.101	0.123	0.131
16	0.075	0.934	0.069	0.067	0.071	0.132	0.819	0.091	0.103	0.115
32	0.072	0.884	0.061	0.062	0.066	0.118	0.793	0.076	0.088	0.099
64	0.040	0.865	0.032	0.034	0.036	0.119	0.744	0.068	0.094	0.105

6.2 Unseen (base는 §6.1과 동일)

blk	base valid	base arr	base $v \wedge a$	base EM	base PC	adj+IW valid	adj+IW arr	adj+IW $v \wedge a$	adj+IW EM	adj+IW PC
1	0.115	0.993	0.114	0.108	0.110	0.932	0.248	0.232	0.145	0.177
2	0.096	0.960	0.092	0.089	0.092	0.243	0.720	0.164	0.174	0.198
4	0.089	0.949	0.084	0.075	0.081	0.168	0.841	0.136	0.141	0.151
8	0.077	0.933	0.071	0.073	0.074	0.145	0.803	0.101	0.124	0.132
16	0.075	0.934	0.069	0.067	0.071	0.134	0.819	0.092	0.102	0.115
32	0.072	0.884	0.061	0.062	0.066	0.117	0.800	0.079	0.084	0.097
64	0.040	0.865	0.032	0.034	0.036	0.121	0.745	0.070	0.095	0.105

- 더 어려운 세계에서 guidance는 유효: 상대 이득은 오히려 커진다(blk64 valid 0.040→0.119, 약 3배; $v \wedge a$ 0.032→0.068-0.070). 다만 절대 수준은 낮다 — 10% 제거에서는 base 모델의 prior와 시나리오 그래프의 불일치가 크다.
- Hard masking의 arrival 비용은 난도와 함께 증가: blk1 adj+IW arrival이 0.25로 붕괴(제거 edge가 2배면 legal-only reveal이 목적지를 지나치는 빈도가 급증). 그래도 $v \wedge a$ 는 base의 2배 (0.114→0.234). v3 보고서처럼 endpoint 후처리(P3)가 자연스러운 보완.
- fam 0.1에서도 zero-shot 전이 성립: 전 블록에서 seen/unseen $v \wedge a$ 격차 ≤ 0.003 .
- adj+IW ESS 97.2(blk64)-99.6(blk1); base < IW < adj+IW 순서 전 블록 유지.

7. 중요도 표본 수 증가 — $n_{is} = 300$ (fam 0.05, except_0, RAW)

§4와 완전히 동일한 프로토콜 — 동일 v4 백본, 동일 v4 판별자, 동일한 except_0 예외 OD 쌍 1,000개, 배치 100, 시드 7, 후처리 없음 — 에서 $n_{is} = 100 \rightarrow 300$ 한 가지만 바꿨다. 괄호 안 흐린 값은 §4의 $n_{is} = 100$ 수치다. base 컬럼은 판별자도 n_{is} 도 쓰지 않고 동일하게 재시도되므로 §4를 그대로 재현하며, 실제로 그렇다는 것이 표본 수 외에는 아무것도 바뀌지 않았다는 내장 검증이 된다. 비용: 경로당 판별자 호출이 $L \cdot 100 \approx 2,200$ 에서 $L \cdot 300 \approx 6,600$ 으로 늘어, §5.6의 guided wall-clock 대비 약 3배다.

7.1 Seen (except_0로 학습한 disc)

blk	base valid	base arr	base v^a	base EM	base PC	adj+IW valid	adj+IW arr	adj+IW v^a	adj+IW EM	adj+IW PC
1	0.302	0.994	0.300	0.271	0.283	0.928 (0.928)	0.566 (0.567)	0.537 (0.540)	0.374 (0.374)	0.425 (0.425)
2	0.251	0.949	0.238	0.233	0.240	0.504 (0.492)	0.824 (0.846)	0.410 (0.401)	0.389 (0.388)	0.428 (0.423)
4	0.279	0.932	0.262	0.243	0.258	0.389 (0.389)	0.872 (0.873)	0.329 (0.328)	0.336 (0.334)	0.358 (0.357)
8	0.229	0.938	0.214	0.202	0.214	0.325 (0.331)	0.869 (0.862)	0.271 (0.273)	0.293 (0.295)	0.306 (0.309)
16	0.206	0.923	0.194	0.180	0.190	0.296 (0.298)	0.856 (0.865)	0.247 (0.250)	0.232 (0.230)	0.256 (0.255)
32	0.185	0.870	0.160	0.157	0.168	0.271 (0.265)	0.828 (0.825)	0.219 (0.215)	0.222 (0.219)	0.241 (0.237)
64	0.141	0.827	0.119	0.129	0.134	0.234 (0.229)	0.782 (0.795)	0.162 (0.162)	0.178 (0.176)	0.200 (0.198)

7.1b IW only, 인접성 마스크 없음 — seen

blk	base valid	base arr	base v^a	base EM	base PC	IW valid	IW arr	IW v^a	IW EM	IW PC
1	0.302	0.994	0.300	0.271	0.283	0.324 (0.324)	0.991 (0.991)	0.321 (0.321)	0.279 (0.283)	0.296 (0.298)
2	0.251	0.949	0.238	0.233	0.240	0.328 (0.328)	0.955 (0.951)	0.311 (0.312)	0.303 (0.306)	0.314 (0.315)
4	0.279	0.932	0.262	0.243	0.258	0.331 (0.332)	0.939 (0.936)	0.305 (0.304)	0.297 (0.295)	0.311 (0.310)
8	0.229	0.938	0.214	0.202	0.214	0.291 (0.294)	0.929 (0.923)	0.257 (0.254)	0.272 (0.270)	0.280 (0.279)
16	0.206	0.923	0.194	0.180	0.190	0.266 (0.260)	0.914 (0.917)	0.240 (0.236)	0.215 (0.211)	0.234 (0.230)
32	0.185	0.870	0.160	0.157	0.168	0.252 (0.244)	0.871 (0.867)	0.209 (0.207)	0.207 (0.204)	0.224 (0.220)
64	0.141	0.827	0.119	0.129	0.134	0.206 (0.201)	0.848 (0.854)	0.157 (0.157)	0.158 (0.160)	0.177 (0.177)

7.2 Unseen (disc = except 1-99 → except_0; base는 §7.1과 동일)

blk	base valid	base arr	base v^a	base EM	base PC	adj+IW valid	adj+IW arr	adj+IW v^a	adj+IW EM	adj+IW PC
1	0.302	0.994	0.300	0.271	0.283	0.928 (0.925)	0.552 (0.558)	0.524 (0.530)	0.365 (0.362)	0.414 (0.412)
2	0.251	0.949	0.238	0.233	0.240	0.495 (0.488)	0.840 (0.848)	0.404 (0.415)	0.374 (0.367)	0.414 (0.408)
4	0.279	0.932	0.262	0.243	0.258	0.389 (0.390)	0.874 (0.875)	0.330 (0.331)	0.333 (0.335)	0.356 (0.357)
8	0.229	0.938	0.214	0.202	0.214	0.333 (0.331)	0.870 (0.863)	0.278 (0.273)	0.299 (0.295)	0.314 (0.308)
16	0.206	0.923	0.194	0.180	0.190	0.299 (0.300)	0.861 (0.868)	0.251 (0.254)	0.238 (0.237)	0.261 (0.261)
32	0.185	0.870	0.160	0.157	0.168	0.274 (0.268)	0.828 (0.828)	0.222 (0.219)	0.226 (0.222)	0.245 (0.240)
64	0.141	0.827	0.119	0.129	0.134	0.235 (0.224)	0.783 (0.798)	0.163 (0.162)	0.179 (0.178)	0.201 (0.197)

7.2b IW only, 인접성 마스크 없음 — unseen

blk	base valid	base arr	base v^a	base EM	base PC	IW valid	IW arr	IW v^a	IW EM	IW PC
1	0.302	0.994	0.300	0.271	0.283	0.321 (0.321)	0.992 (0.992)	0.319 (0.319)	0.279 (0.281)	0.295 (0.296)
2	0.251	0.949	0.238	0.233	0.240	0.331 (0.330)	0.959 (0.951)	0.318 (0.316)	0.305 (0.305)	0.316 (0.316)
4	0.279	0.932	0.262	0.243	0.258	0.330 (0.331)	0.939 (0.935)	0.303 (0.302)	0.295 (0.295)	0.309 (0.310)
8	0.229	0.938	0.214	0.202	0.214	0.300 (0.298)	0.926 (0.923)	0.262 (0.258)	0.278 (0.275)	0.287 (0.284)
16	0.206	0.923	0.194	0.180	0.190	0.267 (0.264)	0.917 (0.919)	0.243 (0.242)	0.217 (0.215)	0.236 (0.234)
32	0.185	0.870	0.160	0.157	0.168	0.253 (0.246)	0.874 (0.869)	0.213 (0.209)	0.211 (0.208)	0.227 (0.223)
64	0.141	0.827	0.119	0.129	0.134	0.204 (0.199)	0.851 (0.857)	0.155 (0.157)	0.158 (0.161)	0.176 (0.177)

7.3 표본 수를 3배로 늘려 얻는 것

- adj+IW, seen: v^a가 블록 평균 +0.001 이동(단일 최대 +0.009), EM은 +0.001(최대 +0.003).
- IW only, seen: v^a 평균 +0.001(최대 +0.004), EM +0.000(최대 +0.004).
- Unseen도 seen과 같은 양상: adj+IW v^a 평균 -0.002, IW only +0.001 — §4.2의 zero-shot 결론은 더 큰 표본 수에서도 그대로다.
- guided 셀 140개 중 0.010을 넘게 움직인 것은 6개뿐이고, 그중 5개가 blk2와 blk64의 adj+IW valid / arrival 쌍이다(최대: seen blk2 arrival -0.022, seen blk2 valid +0.012). 이 블록 크기 들은 §5.6.2가 낙오 행 지배 구간으로 지목한 바로 그 지점으로, 종료하지 않는 소수 행이 배치를 혼돈다 — 즉 남은 변동은 표본 수 효과가 아니라 작은/아주 큰 블록에서 인접성 마스크 샘플러가 갖는 알려진 불안정성이다.

- 유효표본수(ESS)는 포화하지 않고 η_{IS} 에 비례해 늘어난다: 추출 표본 대비 평균 ESS 비율이 $\eta_{IS} = 300$ 에서 blk1 99.6% · blk2 99.1% · blk4 99.2% · blk8 98.5% · blk16 98.2% · blk32 97.7% · blk64 94.7%이고 $\eta_{IS} = 100$ 에서 blk1 99.7% · blk2 99.3% · blk4 99.3% · blk8 98.6% · blk16 98.3% · blk32 97.9% · blk64 95.5%로 동일하다. 즉 절대 ESS가 예상에 비례해 그대로 3배가 됐다. 따라서 가중치 분포가 소수 후보로 집중되는 것이 *아니며*, 유계 로짓($|l| \leq 4$)이 후보 간 비율을 e^8 안으로 묶어두므로 표본을 늘려도 이미 잘 조건화된 추정량의 분산만 줄인다 — 표가 거의 움직이지 않는 이유다.
- 실무적 해석: $\gamma = 1$ 에서 SNIS 추정량은 이 과제에 대해 $\eta_{IS} = 100$ 이면 이미 수렴해 있다. 남은 여지는 표본 예산이 아니라 Lemma 3의 인접성 마스킹(\$4.1 vs \$4.1b)과 판별자 자체에 있다.

8. 판별자 유효성 (요약)

전체 분석은 [DISC_VALIDITY_K0.pdf](#) / [DISC_VALIDITY.pdf](#) . 요약: (i) seen/unseen disc는 held-out 입력에 거의 동일한 점수를 매긴다(logit Pearson 0.81-0.98); (ii) model-negative 과제의 zero-shot AUC 손실은 양방향 probe 모두 ≤ 0.05 — 판별 신호의 대부분(입력 인접행렬 대비 edge 합법성, 모델 artifact)이 구조적으로 시나리오-무관; (iii) 더 어려운 data-negative probe(real vs real)가 두 학습 방식을 가른다: 단일 시나리오 e0 disc는 zero-shot에서 AUC -0.100 (fam 0.05)/ -0.084 (fam 0.1) 하락, 99-시나리오 e99 disc는 거의 평탄 — **시나리오 고 유 정보의 일반화는 다중 시나리오 학습이 만든다**; (iv) v2 blk1의 “최고 AUC”는 퇴화 negative pool의 산물이었고 EOS 수정 후 소멸($0.985 \rightarrow 0.870$) — negative pool 교란 분석의 확증.

9. 블록 내 공개 순서 — left-to-right vs first-hitting (fam 0.05, except_0, RAW)

\$4와 완전히 동일한 프로토콜 — 동일 v4 백본·판별자, 동일한 except_0 예약 OD 쌍 1,000개, 배치 100, 시드 7, $\eta_{IS} = 100$, 후처리 없음 — 에서 **-order l2r** 하나만 바꿨다. mask 커널은 denoising 스텝마다 마스킹된 위치 하나를 확정하는데, **first_hit** (본 보고서의 다른 모든 곳에서 쓴 기본값)은 남은 마스킹 위치 중 하나를 균등 확률로 뽑고 **l2r** 은 항상 가장 왼쪽을 고른다. first-hitting *시간* 스케줄($t \leftarrow t \cdot u^{1/\#masked}$)은 그대로이므로 공개 위치 선택만 분리된다. \$7과 달리 **base** 컬럼도 움직이는데, unguided 계획도 같은 공개 루프를 돌기 때문이다. 괄호 안 흐린 값은 \$4 의 first-hitting 수치다.

노이즈 하한 (blk1): 블록 크기가 1이면 블록당 마스킹 위치가 정확히 하나라 두 방식이 **같은** 위치를 확정한다. 그런데도 행이 동일하지 않은데, **first_hit** 은 위치 선택에 **torch.multinomial** 추출을 한 번 쓰고 **l2r** 은 쓰지 않아 첫 스텝부터 난수 스트림이 갈라지기 때문이다. 따라서 blk1 행은 이 평가 규모에서의 순수 표집 잡음을 측정한다: $v \wedge a + 0.012(\text{base}), +0.007(\text{IW}), +0.002(\text{adj}+\text{IW})$. 나머지 행은 이 ± 0.012 하한에 견주어 읽어야 한다.

9.1 Seen (except_0로 학습한 disc)

blk	base valid	base arr	base $v \wedge a$	base EM	base PC	adj+IW valid	adj+IW arr	adj+IW $v \wedge a$	adj+IW EM	adj+IW PC
1	0.312 (0.302)	0.996 (0.994)	0.312 (0.300)	0.275 (0.271)	0.289 (0.283)	0.925 (0.928)	0.573 (0.567)	0.542 (0.540)	0.361 (0.374)	0.415 (0.425)
2	0.268 (0.251)	0.963 (0.949)	0.262 (0.238)	0.244 (0.233)	0.255 (0.240)	0.867 (0.492)	0.678 (0.846)	0.577 (0.401)	0.440 (0.388)	0.515 (0.423)
4	0.293 (0.279)	0.976 (0.932)	0.285 (0.262)	0.262 (0.243)	0.275 (0.258)	0.765 (0.389)	0.732 (0.873)	0.556 (0.328)	0.434 (0.334)	0.503 (0.357)
8	0.272 (0.229)	0.964 (0.938)	0.256 (0.214)	0.232 (0.202)	0.248 (0.214)	0.760 (0.331)	0.685 (0.862)	0.487 (0.273)	0.392 (0.295)	0.480 (0.309)
16	0.259 (0.206)	0.930 (0.923)	0.240 (0.194)	0.221 (0.180)	0.238 (0.190)	0.747 (0.298)	0.653 (0.865)	0.447 (0.250)	0.389 (0.230)	0.485 (0.255)
32	0.251 (0.185)	0.898 (0.870)	0.221 (0.160)	0.202 (0.157)	0.221 (0.168)	0.723 (0.265)	0.635 (0.825)	0.427 (0.215)	0.376 (0.219)	0.468 (0.237)
64	0.235 (0.141)	0.901 (0.827)	0.202 (0.119)	0.178 (0.129)	0.199 (0.134)	0.828 (0.229)	0.511 (0.795)	0.393 (0.162)	0.359 (0.176)	0.474 (0.198)

9.1b IW only, 인접성 마스킹 없음 — seen

blk	base valid	base arr	base $v \wedge a$	base EM	base PC	IW valid	IW arr	IW $v \wedge a$	IW EM	IW PC
1	0.312 (0.302)	0.996 (0.994)	0.312 (0.300)	0.275 (0.271)	0.289 (0.283)	0.331 (0.324)	0.992 (0.991)	0.328 (0.321)	0.287 (0.283)	0.303 (0.298)
2	0.268 (0.251)	0.963 (0.949)	0.262 (0.238)	0.244 (0.233)	0.255 (0.240)	0.322 (0.328)	0.963 (0.951)	0.317 (0.312)	0.292 (0.306)	0.305 (0.315)
4	0.293 (0.279)	0.976 (0.932)	0.285 (0.262)	0.262 (0.243)	0.275 (0.258)	0.337 (0.332)	0.965 (0.936)	0.320 (0.304)	0.305 (0.295)	0.318 (0.310)
8	0.272 (0.229)	0.964 (0.938)	0.256 (0.214)	0.232 (0.202)	0.248 (0.214)	0.321 (0.294)	0.963 (0.923)	0.303 (0.254)	0.289 (0.270)	0.304 (0.279)
16	0.259 (0.206)	0.930 (0.923)	0.240 (0.194)	0.221 (0.180)	0.238 (0.190)	0.312 (0.260)	0.934 (0.917)	0.281 (0.236)	0.259 (0.211)	0.280 (0.230)
32	0.251 (0.185)	0.898 (0.870)	0.221 (0.160)	0.202 (0.157)	0.221 (0.168)	0.322 (0.244)	0.894 (0.867)	0.267 (0.207)	0.259 (0.204)	0.283 (0.220)

64	0.235 (0.141)	0.901 (0.827)	0.202 (0.119)	0.178 (0.129)	0.199 (0.134)	0.307 (0.201)	0.879 (0.854)	0.247 (0.157)	0.251 (0.160)	0.271 (0.177)
----	---------------	---------------	----------------------	---------------	---------------	---------------	---------------	----------------------	---------------	---------------

9.2 Unseen (disc = except 1-99 → except_0)

blk	base valid	base arr	base v^a	base EM	base PC	adj+IW valid	adj+IW arr	adj+IW v^a	adj+IW EM	adj+IW PC
1	0.312 (0.302)	0.996 (0.994)	0.312 (0.300)	0.275 (0.271)	0.289 (0.283)	0.928 (0.925)	0.556 (0.558)	0.527 (0.530)	0.354 (0.362)	0.406 (0.412)
2	0.268 (0.251)	0.963 (0.949)	0.262 (0.238)	0.244 (0.233)	0.255 (0.240)	0.863 (0.488)	0.677 (0.848)	0.574 (0.415)	0.431 (0.367)	0.507 (0.408)
4	0.293 (0.279)	0.976 (0.932)	0.285 (0.262)	0.262 (0.243)	0.275 (0.258)	0.765 (0.390)	0.738 (0.875)	0.561 (0.331)	0.439 (0.335)	0.506 (0.357)
8	0.272 (0.229)	0.964 (0.938)	0.256 (0.214)	0.232 (0.202)	0.248 (0.214)	0.758 (0.331)	0.690 (0.863)	0.490 (0.273)	0.393 (0.295)	0.481 (0.308)
16	0.259 (0.206)	0.930 (0.923)	0.240 (0.194)	0.221 (0.180)	0.238 (0.190)	0.748 (0.300)	0.653 (0.868)	0.448 (0.254)	0.389 (0.237)	0.485 (0.261)
32	0.251 (0.185)	0.898 (0.870)	0.221 (0.160)	0.202 (0.157)	0.221 (0.168)	0.723 (0.268)	0.633 (0.828)	0.425 (0.219)	0.378 (0.222)	0.469 (0.240)
64	0.235 (0.141)	0.901 (0.827)	0.202 (0.119)	0.178 (0.129)	0.199 (0.134)	0.828 (0.224)	0.510 (0.798)	0.392 (0.162)	0.359 (0.178)	0.474 (0.197)

9.2b IW only, 인접성 마스크 없음 — unseen

blk	base valid	base arr	base v^a	base EM	base PC	IW valid	IW arr	IW v^a	IW EM	IW PC
1	0.312 (0.302)	0.996 (0.994)	0.312 (0.300)	0.275 (0.271)	0.289 (0.283)	0.327 (0.321)	0.992 (0.992)	0.324 (0.319)	0.283 (0.281)	0.299 (0.296)
2	0.268 (0.251)	0.963 (0.949)	0.262 (0.238)	0.244 (0.233)	0.255 (0.240)	0.328 (0.330)	0.966 (0.951)	0.321 (0.316)	0.298 (0.305)	0.311 (0.316)
4	0.293 (0.279)	0.976 (0.932)	0.285 (0.262)	0.262 (0.243)	0.275 (0.258)	0.336 (0.331)	0.972 (0.935)	0.326 (0.302)	0.303 (0.295)	0.316 (0.310)
8	0.272 (0.229)	0.964 (0.938)	0.256 (0.214)	0.232 (0.202)	0.248 (0.214)	0.323 (0.298)	0.963 (0.923)	0.304 (0.258)	0.290 (0.275)	0.305 (0.284)
16	0.259 (0.206)	0.930 (0.923)	0.240 (0.194)	0.221 (0.180)	0.238 (0.190)	0.315 (0.264)	0.934 (0.919)	0.283 (0.242)	0.263 (0.215)	0.284 (0.234)
32	0.251 (0.185)	0.898 (0.870)	0.221 (0.160)	0.202 (0.157)	0.221 (0.168)	0.322 (0.246)	0.896 (0.869)	0.269 (0.209)	0.263 (0.208)	0.285 (0.223)
64	0.235 (0.141)	0.901 (0.827)	0.202 (0.119)	0.178 (0.129)	0.199 (0.134)	0.306 (0.199)	0.879 (0.857)	0.248 (0.157)	0.250 (0.161)	0.270 (0.177)

9.3 해석

- 블록 크기 2 이상에서, 모든 구성에서 left-to-right이 이긴다. v^a 이득(seen): base와 IW only는 블록 크기에 따라 단조 증가하고(base blk2 +0.024 → blk64 +0.083; IW only +0.005 → +0.090), adj+IW는 블록 2 이상 전 구간에서 크고 대체로 평평하다(blk 2...64에서 +0.176, +0.228, +0.214, +0.197, +0.212, +0.231). blk64에서 adj+IW v^a가 0.162 → 0.393으로 2.4배가 되는데, 비용은 전혀 들지 않는 변경이다.
- adj+IW의 이득은 valid/arrival 맞교환이지만 결과적으로 크게 남는 장사다. validity가 두 배 안팎으로 뛰고(blk4 0.389 → 0.765; blk64 0.229 → 0.828) arrival은 떨어진다(blk4 0.873 → 0.732; blk64 0.795 → 0.511). 그럼에도 곱은 각각 +0.228, +0.231 상승하고, 맞교환에 다칠 수 있었던 EM/PC도 함께 오른다(blk64에서 EM +0.183, PC +0.276).
- Lemma-3 마스크링과의 상호작용이 큰 이유: 인접성 마스크는 대상 위치의 이웃이 이미 공개된 경우에만 후보를 제약할 수 있다 — 좌우 이웃이 아직 MASK면 코드가 전부 1인 마스크로 되돌아간다. l2r에서는 왼쪽 이웃이 구조적으로 매 스텝 공개돼 있으므로(직전 스텝에서 확정됐다) 모든 확정이 실제 prefix의 적법한 연장으로 제한된다. first_hit에서는 좌우가 모두 마스크링된 상태로 확정되는 위치가 많고, 그 토큰들은 아무 제약 없이 놓인 뒤 나중 공개가 그것에 맞춰야 한다 — invalid edge가 여기서 나온다. adj+IW에서 효과가 IW only나 base보다 훨씬 큰 이유다.
- guidance만의 이야기가 아니다: base와 IW only도 이득을 보고 그 이득 역시 블록 크기에 따라 커진다(base blk8 +0.042 → blk64 +0.083). 측정이 아닌 해석을 붙이자면: 왼쪽부터 공개하면 denoiser에게 묻는 모든 조건부가 연속된 완전한 prefix를 조건으로 갖는데, 이것이 도로망 경로가 실제로 가진 구조다. 출어진 문맥은 더 어려운 질의이고, 큰 블록일수록 그런 스텝이 많아 틀릴 여지가 크다 — 이득이 단조 증가하는 것과 일치한다.
- zero-shot 일반화는 그대로다: l2r 표 전체에서 seen-unseen 최대 차이가 \$4.2와 같은 수준이다. 공개 순서는 판별자가 어느 시나리오로 학습했는지와 상호작용하지 않는다.
- 보고서 전체 최고 셀 경신: adj+IW l2r blk2가 v^a 0.577(seen) / 0.574(unseen)로, 종전 최고 0.540(blk1 adj+IW, \$4.1)을 넘는다. 게다가 blk1과 달리 쓸 만한 arrival(0.678 vs 0.567)을 유지한다.
- 비용: 없음. l2r은 스텝당 multinomial 추출을 하나 없애고 채점 호출 수는 바꾸지 않으므로 \$5.6의 wall-clock 표가 그대로 적용된다. n_g = 100에서 평균 ESS는 blk1 99.7 · blk2 98.4 · blk4 98.3 · blk8 97.6 · blk16 95.8 · blk32 95.6 · blk64 94.9로 first-hitting(99.7 ... 95.5)보다 근소하게 낮다 — 가중치가 오히려 조금 더 퍼진다.
- 주의: 칸마다 시드 하나다. blk1 행이 v^a 잡음을 ≈0.012로 묶어주므로 blk8 이상의 이득(0.04-0.23)은 확실히 그 밖이지만, blk2/blk4의 base-IW only 이득(0.005-0.024)은 하한에 걸쳐 있어 개별적으로 확립된 것으로 읽으면 안 된다 — 일곱 개 블록에 걸친 단조 추세만이 근거다.

10. 비고 · 출처

- 본 보고서의 모든 수치는 RAW다(P1/P3 없음, refine 없음 — conditional 스킴은 `use_refine=False`). §4-§5-§6의 후처리 대응 결과는 별도 파일 `RESULTS_BD_V4_POSTPROC_K0.pdf` 에 있다.
- D-CBG 라운드(§5): `run_v4_dcbg_train.sh` (분류기 28개, model-negative만) → `run_v4_dcbg_eval.sh` (평가 28개, exact + first-order, $\gamma=1$) 및 `time_guidance.py` ; 산출물 `sets_res/dcbg_table_v4_f005.json` , `guidance_timing_v4.json` .
- 파이프라인: `run_v4_all.sh` , 단일 tmux 체인, 2026-07-27 00:48-04:22 KST (Stage A 재학습 $\approx 1\text{h}40\text{m}$; guided 평가 28개 $\approx 1\text{h}$). 원자료: `sets_res/va_table_v4_f005.json` , `va_table_v4_f01.json` , `uncond_pool_eval_v4.json` , `disc_gen_eval_f{005,01}{,_data}_v4.json` , 로그 `sets_log/v4_*.log` .
- 학습은 캔버스 수정 외 v2와 동일: `train_porto_bd.sh mask B gpu v4` (1 epoch, bs 32, lr $5\text{e}-4$, drop_cond 0.1 — ori/dst는 항상 *동시*에 drop, 개별 drop 없음).
- conditional 평가 스크립트의 출력 prefix가 `V2_` 로 하드코딩되어 해당 per-path CSV는 현재 v4 결과를 담는다 (v2 집계는 `sets_log/v2_ctrl_mask.log` 와 v3 보고서에 보존).
- **s/path 주의**: first-hitting에서 알고리즘적 NFE는 블록 크기와 무관하다(reveal당 forward 1회, 총합 $\approx$ 생성 길이). 측정된 blk1의 느림(0.082 vs ~ 0.033 s/path)은 블록당 구현 오버헤드 — CPU 동기화 + 파이썬 정지 판정 루프 + attention mask 재구축을 B=1에서는 *토큰마다* 지불 — 와 배치 straggler 동기화 때문이며, 큰 블록의 알고리즘적 이점이 아니다 (KV 캐시·블록 내 병렬 확장 미구현).
- 수정 전 체크포인트/disc/pool은 그대로 보존(모든 v4 산출물이 별도 이름). 본 보고서 확정 후 삭제 가능.